

SLOT3:

EXPRESSJS, Nosql Databases With Mongodb

Tóm tắt nội dung

EXPRESSJS:

7.1 Express Introduction

7.2 Using Express js to create your first program

NODE and EXPRESS:

8.1 Restful routing with express

8.2 Understand and use middleware for node applications

8.3 Types of middleware in node.js

8.4 Writing express middleware

8.5 Express- Generator

- Express Generator Introduction

- Using Express - Generator To Create Your First Program

Nosql Databases With Mongodb:

9.1 Nosql Introduction

9.2 Document Databases (Mongodb)

9.3 Create-Read-Update-Destroy (Crud)

7. EXPRESSJS

7.1 Express Introduction

- Giới thiệu về Express.js – một framework web tối giản, linh hoạt cho Node.js.
- Hiểu lý do tại sao nên sử dụng Express trong các ứng dụng web backend.

7.2 Using Express.js to Create Your First Program

- Thiết lập dự án đầu tiên với Express.
- Viết ứng dụng web cơ bản (ví dụ: hiển thị “Hello World”).

8. NODE AND EXPRESS

8.1 Restful Routing with Express

- Hiểu khái niệm RESTful và cách xây dựng các tuyến đường (routes) trong Express.
- Tạo các route như GET, POST, PUT, DELETE để thao tác dữ liệu.

8.2 Understand and Use Middleware for Node Applications

- Tìm hiểu về middleware: đoạn mã chạy giữa quá trình request và response.
- Cách sử dụng middleware để xử lý logic chung (vd: logging, kiểm tra quyền truy cập).

8.3 Types of Middleware in Node.js

- Phân loại middleware: built-in, third-party, custom.
- Ví dụ các loại phổ biến như body-parser, logger, error handler.

8.4 Writing Express Middleware

- Tự viết middleware tùy chỉnh.
- Sử dụng `next()` để chuyển tiếp request qua nhiều middleware.

8.5 Express Generator

Express Generator Introduction

- Giới thiệu công cụ Express Generator để tạo cấu trúc dự án nhanh chóng.

Using Express Generator to Create Your First Program

- Cài đặt Express Generator.
- Tạo ứng dụng mẫu và hiểu cấu trúc thư mục được tạo tự động.

9. NOSQL DATABASES WITH MONGODB

9.1 NoSQL Introduction

- Phân biệt NoSQL với cơ sở dữ liệu quan hệ (SQL).
- Ưu điểm của NoSQL (linh hoạt, mở rộng tốt...).

9.2 Document Databases (MongoDB)

- Giới thiệu MongoDB – cơ sở dữ liệu dạng tài liệu (document-based).
- Hiểu cách lưu trữ dữ liệu dưới dạng JSON (BSON).

9.3 Create-Read-Update-Destroy (CRUD)

- Thao tác dữ liệu cơ bản với MongoDB: Tạo, Đọc, Cập nhật, Xóa.
- Kết nối Express và MongoDB để thực hiện CRUD qua API.

✓ CÁC NỘI DUNG TRỌNG TÂM, CỐT LÕI CẦN LƯU Ý:

7. EXPRESSJS

✓ 7.1 Express Introduction

- **Express = Web framework cho Node.js**
- Cung cấp công cụ xây dựng **API** và **ứng dụng web**
- Dễ dùng, nhanh, linh hoạt hơn so với Node.js thuần

✓ 7.2 Using Express.js to Create Your First Program

- Biết cách tạo **ứng dụng Express đầu tiên**
- Dùng `npm init`, `npm install express`, tạo file `app.js`
- Sử dụng `app.get('/', callback)` để tạo một route cơ bản
- Khởi chạy server bằng `app.listen(port)`

◆ 8. NODE AND EXPRESS

✅ 8.1 Restful Routing with Express

- RESTful là quy ước xây dựng **API CRUD**:
 - GET – Lấy dữ liệu
 - POST – Tạo dữ liệu mới
 - PUT / PATCH – Cập nhật dữ liệu
 - DELETE – Xóa dữ liệu
- Express cung cấp phương thức tương ứng: `app.get`, `app.post`, `app.put`, `app.delete`

✅ 8.2 Understand and Use Middleware for Node Applications

- **Middleware** = Hàm xử lý request trước khi response trả về
- Dùng để:
 - Xử lý dữ liệu (`body-parser`)
 - Logging
 - Kiểm tra xác thực (`auth`)

✅ 8.3 Types of Middleware in Node.js

- **Built-in**: Có sẵn trong Express (ví dụ: `express.static`)
- **Third-party**: Thư viện cài thêm (vd: `morgan`, `cors`)
- **Custom**: Tự viết, dùng cho logic riêng

✅ 8.4 Writing Express Middleware

- Cách viết middleware: `(req, res, next) => { ... }`
- Gọi `next()` để chuyển sang middleware kế tiếp
- Có thể xử lý lỗi với middleware đặc biệt có 4 tham số (`err, req, res, next`)

✅ 8.5 Express Generator

- Công cụ CLI tạo dự án Express sẵn cấu trúc thư mục
- Cài đặt: `npm install -g express-generator`
- Tạo project: `express myapp`
- Chạy thử với `npm install + npm start`

◆ 9. NOSQL DATABASES WITH MONGODB

✅ 9.1 NoSQL Introduction

- Không dùng bảng dòng/cột như SQL
- Dữ liệu lưu dưới dạng **document (JSON/BSON)**
- NoSQL phù hợp với hệ thống **linh hoạt, mở rộng nhanh**

✅ 9.2 Document Databases (MongoDB)

- MongoDB lưu dữ liệu theo **collection** → **document**

- Mỗi document là một object JSON
- Linh hoạt trong lưu trữ: không cần schema cố định

✓ 9.3 CRUD (Create - Read - Update - Delete)

- Kỹ năng cốt lõi:
 - **Create:** `db.collection.insertOne()`
 - **Read:** `db.collection.find()`
 - **Update:** `db.collection.updateOne()`
 - **Delete:** `db.collection.deleteOne()`
- Thực hành thao tác MongoDB từ Express qua thư viện `mongoose` hoặc `mongodb`

Tóm lại – Những điểm CẦN GHI NHỚ

Chủ đề	Nội dung cốt lõi
ExpressJS	Biết tạo ứng dụng web/API đơn giản
Routing	RESTful API với GET, POST, PUT, DELETE
Middleware	Biết loại, cách viết và sử dụng
Express Generator	Tạo dự án nhanh có cấu trúc thư mục sẵn
NoSQL & MongoDB	Dữ liệu lưu kiểu document, không có bảng
CRUD	Thành thạo 4 thao tác cơ bản với MongoDB

✓ ỨNG DỤNG THỰC TIỄN:

Kỹ thuật	Ứng dụng thực tiễn
ExpressJS	Viết backend/API cho web và mobile app
RESTful routing	Tạo API chuẩn để kết nối React Native, ReactJS
Middleware	Auth, xử lý lỗi, log, bảo mật
Express Generator	Khởi tạo project nhanh, phù hợp teamwork
MongoDB NoSQL	Lưu dữ liệu dạng document: user, post, sản phẩm
CRUD với MongoDB	Mọi thao tác dữ liệu backend: Tạo – Đọc – Sửa – Xóa

◆ 7. EXPRESSJS

✓ Ứng dụng thực tiễn:

- **Xây dựng API cho ứng dụng di động React Native**
 - Ví dụ: API /login, /register, /tasks, /profile
- **Tạo website backend** như hệ thống quản lý sinh viên, blog cá nhân
- **Quản lý request/response nhanh gọn** mà không cần code nhiều như Node.js thuần

◆ 8. NODE + EXPRESS

✓ 8.1 RESTful Routing

- **Tạo các API chuẩn REST** để làm việc với frontend (React Native, web)
 - Ví dụ: App To-do list có các route /todos (GET, POST), /todos/:id (PUT, DELETE)
- Giúp tổ chức code backend logic **rõ ràng, có quy tắc**

✓ 8.2 - 8.4 Middleware

- **Xác thực người dùng (Authentication):** Middleware kiểm tra token JWT
- **Xử lý lỗi tập trung:** Middleware thu thập lỗi thay vì xử lý riêng từng chỗ
- **Ghi log, đo thời gian phản hồi API**
- **Nén dữ liệu, xử lý CORS,** tiền xử lý dữ liệu gửi về từ frontend

✓ 8.5 Express Generator

- Dùng để tạo project backend nhanh có cấu trúc chuẩn: giúp dễ bảo trì
- Tăng tốc độ làm việc nhóm khi dự án backend có nhiều tệp và thư mục phức tạp

◆ 9. NOSQL DATABASES WITH MONGODB

✓ 9.1 NoSQL Introduction

- Thích hợp với hệ thống cần **mở rộng lớn, lưu dữ liệu không cố định**
 - Ví dụ: App thương mại điện tử với sản phẩm đa dạng thông tin

✓ 9.2 Document Databases (MongoDB)

- Lưu trữ dữ liệu người dùng, đơn hàng, bài viết, comment...
- Kết hợp dễ dàng với Express và Node để tạo hệ thống lưu trữ động
- Dùng trong các hệ thống **realtime**, mạng xã hội, blog, app học tập...

✓ 9.3 CRUD

- **Cốt lõi cho mọi ứng dụng backend**
 - Tạo user, cập nhật profile, đọc danh sách task, xóa comment,...
- Tất cả các app thực tế đều cần logic CRUD phía backend và lưu vào MongoDB

CHI TIẾT MỘT SỐ NỘI DUNG:

7. EXPRESSJS – Giới thiệu và xây dựng ứng dụng đầu tiên

Mục tiêu:

Nắm được nền tảng Express – framework phổ biến nhất dùng với Node.js để xây dựng web server và REST API.

7.1 Express Introduction

Giải thích chuyên sâu:

- Express là một **framework nhẹ** cho Node.js, giúp xử lý HTTP nhanh gọn.
- Express không thay thế Node, mà **làm cho Node dễ dùng hơn**, bằng cách:
 - Đơn giản hóa việc tạo HTTP server
 - Cho phép dễ dàng xử lý route, middleware
 - Cho phép chia nhỏ ứng dụng (routing, controller...)

Ví dụ:

js

```
const express = require('express');
const app = express();
app.get('/', (req, res) => res.send('Hello World!'));
app.listen(3000);
```

7.2 Using Express.js to Create Your First Program

Giải thích chuyên sâu:

- Tạo ứng dụng Express đầu tiên:
 - Cài đặt bằng `npm install express`
 - Tạo server: `app.listen(PORT)`
 - Tạo các routes: `app.get/post/put/delete(...)`
- **Request:** thông tin gửi từ client
- **Response:** dữ liệu server trả về

Thực hành:

- Tạo một route `/hello` trả về JSON
- Gắn thêm route `/api/users` để trả về danh sách người dùng

◆ 8. NODE & EXPRESS – Xây dựng API chuẩn, dùng middleware, tạo cấu trúc dự án

🎯 Mục tiêu:

Hiểu được cách xây dựng **RESTful API**, dùng **middleware**, và **cấu trúc hóa** backend để phục vụ cho các ứng dụng frontend như React Native.

✅ 8.1 RESTful Routing with Express

Giải thích chuyên sâu:

- REST (Representational State Transfer) là quy tắc tổ chức API
- Mỗi route + HTTP verb đại diện cho hành động:
 - GET /users – lấy tất cả users
 - POST /users – tạo mới
 - PUT /users/:id – cập nhật
 - DELETE /users/:id – xóa

Lưu ý:

- Sử dụng route logic tách biệt file (`routes/userRoutes.js`)
- Giúp frontend dễ tích hợp, backend dễ maintain

✅ 8.2 – 8.4 Middleware trong Express

Giải thích chuyên sâu:

- **Middleware** = logic trung gian giữa request và response
- Gọi `next()` để chuyển tiếp đến middleware tiếp theo
- Dùng để:
 - Parse dữ liệu (`body-parser`, `express.json`)
 - Logging request (`morgan`)
 - Kiểm tra token/authorization
 - Bắt lỗi toàn hệ thống (`error handler`)

Ví dụ:

```
js
app.use((req, res, next) => {
  console.log("Middleware chạy trước route");
  next();
});
```

✓ 8.5 Express Generator

Giải thích chuyên sâu:

- Công cụ CLI tự động tạo:
 - routes/, views/, app.js, public/
 - Giúp bạn không cần setup thủ công ban đầu
- Thích hợp khi làm việc nhóm, hoặc khi dự án phức tạp

Lệnh cần nhớ:

bash

```
npm install -g express-generator
express myapp
cd myapp
npm install
npm start
```

◆ 9. NoSQL & MongoDB – Lưu trữ linh hoạt và thao tác dữ liệu

🎯 Mục tiêu:

Hiểu và sử dụng cơ bản MongoDB để lưu trữ dữ liệu dạng document, thực hiện các thao tác CRUD.

✓ 9.1 NoSQL Introduction

Giải thích chuyên sâu:

- NoSQL: Not Only SQL – phù hợp với dữ liệu phi cấu trúc
- **Không có bảng – dòng – cột** như SQL
- Ưu điểm:
 - Linh hoạt schema
 - Mở rộng tốt (scalable)
 - Dễ làm việc với dữ liệu JSON – phù hợp với JS/Node/React Native

✓ 9.2 Document Databases (MongoDB)

Giải thích chuyên sâu:

- MongoDB lưu data dưới dạng **document JSON** trong **collection**
- Cấu trúc dữ liệu không cần cứng nhắc, ví dụ:

json

```
{
  "_id": "1",
  "name": "John",
  "email": "john@gmail.com"
}
```

- Mỗi document có thể khác nhau về schema → linh hoạt

✓ 9.3 CRUD (Create - Read - Update - Delete)

Giải thích chuyên sâu:

- Cần học **thao tác MongoDB qua thư viện mongoose** hoặc `mongodb-native`
- Các thao tác chính:
 - `Model.create(data)` → Tạo document
 - `Model.find(query)` → Lấy dữ liệu
 - `Model.updateOne(query, newData)` → Cập nhật
 - `Model.deleteOne(query)` → Xóa

Ví dụ với Mongoose:

js

```
const User = mongoose.model("User", { name: String, email: String });
User.find().then(users => console.log(users));
```

✓ LỘ TRÌNH GỢI Ý (kết hợp React Native)

Mục tiêu	Backend hỗ trợ
Đăng ký / Đăng nhập	Express + MongoDB + JWT Auth
Quản lý user/profile	CRUD API dùng Express
Lưu trữ ghi chú / task	MongoDB + REST API
Tổ chức code tốt	Express Generator + Middleware

KINH NGHIỆM THỰC TIỄN, VẤN ĐỀ CẦN LƯU Ý, VÀ CÁC MẸO (TIPS)

1. KINH NGHIỆM THỰC TẾ

ExpressJS & Node.js

Kinh nghiệm	Mô tả
Tách biệt các lớp	Luôn tách <code>routes</code> , <code>controllers</code> , <code>services</code> , <code>models</code> để dễ bảo trì và mở rộng
Dùng <code>dotenv</code>	Cấu hình <code>PORT</code> , <code>DB_URI</code> , <code>SECRET_KEY</code> trong <code>.env</code> , tránh <code>hard-code</code>
Dùng <code>Postman</code> hoặc <code>Insomnia</code>	Để test API hiệu quả trước khi tích hợp frontend
Logging	Sử dụng <code>morgan</code> để ghi log HTTP request giúp debug dễ dàng
Validate dữ liệu	Dùng thư viện như <code>express-validator</code> hoặc <code>Joi</code> để kiểm tra dữ liệu đầu vào
Dùng <code>async/await</code>	Viết code <code>async</code> sạch sẽ, tránh <code>callback hell</code>

MongoDB

Kinh nghiệm	Mô tả
Dùng <code>Mongoose</code>	Giúp dễ thao tác hơn <code>mongodb-native</code> , hỗ trợ <code>schema</code> , <code>validate</code>
Kiểm tra <code>connection error</code>	Bắt lỗi kết nối với MongoDB sớm, in log rõ ràng
Index	Với dữ liệu lớn, cần tạo index để tăng tốc độ truy vấn
Hạn chế <code>nested document</code>	Tránh lồng sâu nhiều lớp – khó đọc và cập nhật
Đặt tên rõ ràng	Collection nên đặt ở dạng số nhiều (<code>users</code> , <code>products</code> , ...), field ngắn gọn

2. CÁC VẤN ĐỀ THƯỜNG GẶP & CÁCH XỬ LÝ

Vấn đề	Nguyên nhân	Giải pháp
Không kết nối MongoDB	Thiếu <code>.env</code> , sai URI, chưa bật MongoDB	Kiểm tra <code>MONGO_URI</code> , dùng <code>Compass test</code>
API trả về <code>undefined</code>	Sai <code>req.body</code> , thiếu middleware	Dùng <code>express.json()</code> để parse JSON
Lỗi CORS khi frontend gọi API	Backend không cho phép domain frontend	Thêm <code>cors middleware</code> (<code>app.use(cors())</code>)
Quên dùng <code>next()</code> trong middleware	Không chuyển tiếp sang bước tiếp theo	Luôn gọi <code>next()</code> hoặc xử lý lỗi đúng
Truy vấn MongoDB chậm	Collection lớn, không có index	Dùng <code>.explain()</code> để phân tích truy vấn

3. MẸO (TIPS) GIÚP LÀM VIỆC NHANH VÀ HIỆU QUẢ

ExpressJS & Node

- Dùng `nodemon` để tự động restart server khi code thay đổi (`npm install nodemon -D`)
- Gắn prefix `/api/` cho các route để dễ quản lý (`/api/users`, `/api/products`)
- Tạo middleware bắt lỗi toàn cục:

js

```
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ error: "Something went wrong" });
});
```

MongoDB & Mongoose

- Tạo schema với validate rõ ràng:

js

```
const userSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true },
  age: { type: Number, min: 18 }
});
```

- Sử dụng `.lean()` khi chỉ cần đọc dữ liệu (trả về plain object → nhanh hơn):

js

```
User.find().lean().then(data => ...)
```

- Sử dụng `.select('-password')` để ẩn dữ liệu nhạy cảm khi gửi về client

Tích hợp React Native

- Dùng thư viện như `axios` để gọi API:

js

```
axios.get('http://localhost:3000/api/users')
```

- Khi chạy trên thiết bị thật (Android, iOS), cần thay `localhost` thành IP thật của backend (`http://192.168.x.x:3000`)
- Nếu backend dùng port khác 80/443, phải đảm bảo port đó mở trong tường lửa / router

Checklist Tự Kiểm Tra Dự Án

☒	Mục cần kiểm tra
☐	API RESTful đầy đủ CRUD
☐	Dữ liệu validate đầy đủ
☐	Tách biệt rõ route, controller, model
☐	Log rõ ràng, dễ debug
☐	Đảm bảo bảo mật dữ liệu nhạy cảm (ẩn mật khẩu, token)
☐	Xử lý lỗi (try-catch hoặc middleware)
☐	React Native gọi API thành công và hiển thị dữ liệu

EXERCISE

"Task Manager API"

✓ 1. MỤC TIÊU DỰ ÁN

- Xây dựng RESTful API để quản lý danh sách công việc (tasks).
- Thực hành CRUD (Create, Read, Update, Delete).
- Hiểu cách dùng Express, Middleware, MongoDB, Mongoose.
- Làm backend API có thể tích hợp với frontend như React Native sau này.

✓ 2. CÁC CHỨC NĂNG

STT	Chức năng	Mô tả
1	Tạo task	Người dùng tạo task mới
2	Lấy danh sách	Truy vấn danh sách tất cả task
3	Lấy 1 task	Truy vấn task theo ID
4	Cập nhật task	Cập nhật thông tin task
5	Xóa task	Xóa task theo ID

✓ 3. CÔNG NGHỆ SỬ DỤNG

- **Node.js + Express.js:** Tạo server API.
- **MongoDB:** Cơ sở dữ liệu NoSQL.
- **Mongoose:** ODM để thao tác Mongo dễ dàng.
- **Postman:** Test API.
- **Nodemon:** Tự reload server khi code thay đổi.
- **dotenv:** Quản lý biến môi trường.

✓ 4. CẤU TRÚC THƯ MỤC DỰ ÁN

arduino

```
task-manager-api/  
├── config/  
│   └── db.js  
├── controllers/  
│   └── taskController.js  
├── models/  
│   └── Task.js  
├── routes/  
│   └── taskRoutes.js  
├── middleware/  
│   └── errorHandler.js  
├── .env  
├── server.js  
└── package.json
```

✓ 5. TÓM TẮT CÁC BƯỚC THỰC HIỆN

1. Khởi tạo project và cài đặt thư viện.
2. Kết nối MongoDB (dùng Mongoose).
3. Tạo schema và model `Task`.
4. Tạo controller để xử lý logic CRUD.
5. Tạo route Express cho từng chức năng.
6. Tạo middleware xử lý lỗi.
7. Tích hợp mọi thứ vào `server.js`.
8. Test với Postman.

✓ 6. HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC

◆ Bước 1: Khởi tạo project và cài đặt thư viện

```
bash

mkdir task-manager-api
cd task-manager-api
npm init -y
npm install express mongoose dotenv cors
npm install --save-dev nodemon
```

Tạo file cấu hình `package.json` script:

```
json

"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
}
```

◆ Bước 2: Cấu hình MongoDB

Tạo file `.env`

```
env

PORT=5000
MONGO_URI=mongodb://localhost:27017/taskdb
```

Tạo `config/db.js`:

```
js

const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB connected");
  } catch (err) {
```

```

        console.error(err.message);
        process.exit(1);
    }
};

module.exports = connectDB;

```

◆ Bước 3: Tạo model **Task**

File models/Task.js

```

js

const mongoose = require('mongoose');

const TaskSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
  },
  completed: {
    type: Boolean,
    default: false,
  },
}, { timestamps: true });

module.exports = mongoose.model('Task', TaskSchema);

```

◆ Bước 4: Tạo controller **CRUD**

File controllers/taskController.js

```

js

const Task = require('../models/Task');

// Create
exports.createTask = async (req, res) => {
  const task = new Task(req.body);
  await task.save();
  res.status(201).json(task);
};

// Read all
exports.getTasks = async (req, res) => {
  const tasks = await Task.find();
  res.json(tasks);
};

// Read one
exports.getTask = async (req, res) => {
  const task = await Task.findById(req.params.id);
  if (!task) return res.status(404).json({ error: "Task not found" });
  res.json(task);
};

// Update
exports.updateTask = async (req, res) => {

```

```

    const task = await Task.findByIdAndUpdate(req.params.id, req.body, { new:
true });
    res.json(task);
  };

// Delete
exports.deleteTask = async (req, res) => {
  await Task.findByIdAndDelete(req.params.id);
  res.json({ message: "Task deleted" });
};

```

◆ Bước 5: Tạo routes

File routes/taskRoutes.js

```

js

const express = require('express');
const router = express.Router();
const taskController = require('../controllers/taskController');

router.post('/', taskController.createTask);
router.get('/', taskController.getTasks);
router.get('/:id', taskController.getTask);
router.put('/:id', taskController.updateTask);
router.delete('/:id', taskController.deleteTask);

module.exports = router;

```

◆ Bước 6: Middleware xử lý lỗi

File middleware/errorHandler.js

```

js

module.exports = (err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Something went wrong!' });
};

```

◆ Bước 7: Tạo server.js

```

js

const express = require('express');
const connectDB = require('./config/db');
const dotenv = require('dotenv');
const cors = require('cors');
const errorHandler = require('./middleware/errorHandler');

dotenv.config();
connectDB();

const app = express();
app.use(cors());

```

```
app.use(express.json());

app.use('/api/tasks', require('./routes/taskRoutes'));
app.use(errorHandler);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

◆ Bước 8: Chạy và test dự án

bash

npm run dev

Dùng **Postman** để test các endpoint:

- POST /api/tasks – tạo task
- GET /api/tasks – lấy danh sách
- GET /api/tasks/:id – lấy 1 task
- PUT /api/tasks/:id – cập nhật
- DELETE /api/tasks/:id – xóa task

ĐÃ HOÀN THÀNH MỤC TIÊU!

-  Áp dụng được ExpressJS, MongoDB, Middleware
-  Hiểu rõ cách cấu trúc dự án backend thực tế
-  Có thể dùng backend này để kết nối với ứng dụng React Native sau

PHÂN TÍCH QUY LUẬT THỰC THI

PROMT: Tôi là người mới học, tôi cần phát hiện quy luật thực thi của chương trình. Hãy Mô tả quá trình thực thi của code của bài tập

Chúng ta sẽ mô tả quá trình thực thi của chương trình theo **thứ tự dòng chảy**, từ lúc chạy `node server.js` đến khi người dùng gọi API như `GET /api/tasks`.

Giai đoạn 1: Khởi động ứng dụng (server.js)

◆ Khi bạn chạy `node server.js` hoặc `npm run dev`:

```
js
```

```
const express = require('express');  
const mongoose = require('mongoose');  
const app = express();
```

- Tạo ra ứng dụng **Express** để làm Web server.
- Kết nối tới **MongoDB** bằng thư viện `mongoose`.

```
js
```

```
app.use(express.json());
```

- Kích hoạt middleware `express.json()` để **đọc dữ liệu JSON** gửi từ client.

```
js
```

```
app.use('/api/tasks', require('./routes/taskRoutes'));
```

- Định nghĩa: Nếu URL bắt đầu bằng `/api/tasks`, thì chuyển đến file `taskRoutes.js`.

```
js
```

```
app.listen(PORT, () => console.log(...));
```

- Mở server lắng nghe yêu cầu (requests) tại `http://localhost:5000`.

Giai đoạn 2: Routing – khi người dùng gọi `GET /api/tasks`

Giả sử người dùng mở Postman và gửi `GET http://localhost:5000/api/tasks`

```
js
```

```
// routes/taskRoutes.js  
router.get('/', getTasks);
```

- Khi thấy `GET /api/tasks`, Express gọi hàm `getTasks` từ `taskController.js`.

Giai đoạn 3: Xử lý controller

js

```
// controllers/taskController.js
const getTasks = async (req, res) => {
  const tasks = await Task.find();
  res.json(tasks);
};
```

1. Hàm này gọi `Task.find()` để lấy toàn bộ dữ liệu từ MongoDB.
2. Kết quả (mảng các task) được gửi trả lại qua `res.json(...)`.

Giai đoạn 4: Kết nối cơ sở dữ liệu (db.js và mongoose.connect)

js

```
mongoose.connect(process.env.MONGO_URI)
```

- Kết nối tới MongoDB bằng chuỗi `MONGO_URI` từ file `.env`.
- Nếu thành công, ứng dụng mới tiếp tục lắng nghe request.

Tóm tắt luồng thực thi tổng quát

Giai đoạn	Diễn giải
1. Chạy file <code>server.js</code>	Tạo server Express, kết nối MongoDB, load router.
2. Gửi yêu cầu (request)	Ví dụ: gửi <code>GET /api/tasks</code> bằng Postman.
3. Express phân tích route	Tìm thấy route <code>/api/tasks</code> → gọi <code>getTasks</code> .
4. Gọi controller <code>getTasks</code>	Truy vấn MongoDB lấy danh sách task.
5. Trả dữ liệu về client	Dữ liệu được trả về dạng JSON qua <code>res.json()</code> .

BÀI TẬP 1: Hello Express

Mục đích

Làm quen với môi trường ExpressJS và cách tạo một web server cơ bản.

Tóm tắt nội dung

Tạo một ứng dụng NodeJS sử dụng Express, hiển thị thông điệp “Hello Express” khi truy cập trình duyệt.

Yêu cầu

- Tạo một web server với Express.
- Xử lý HTTP GET request tại /.

Các chức năng

- GET / → trả về chuỗi "Hello Express".

Công nghệ sử dụng

- Node.js
- Express.js

Cấu trúc dự án

```
hello-express/  
├── package.json  
└── server.js
```

Các bước thực hiện

- Tạo thư mục dự án hello-express.
- Khởi tạo npm: `npm init -y`.
- Cài Express: `npm install express`.
- Tạo `server.js` và viết code khởi động server.
- Test bằng trình duyệt hoặc Postman.

Kết quả cần đạt được

- Tạo được một server Express đơn giản.
- Truy cập / sẽ nhận được chuỗi "Hello Express".

Test case:

1. Gửi HTTP GET request tới `http://localhost:3000/` → Nhận kết quả: Hello Express.
2. Truy cập / bằng trình duyệt web → Trang hiển thị đúng dòng chữ: Hello Express.

BÀI TẬP 2: Quotes API (Dữ liệu tĩnh)

Mục đích

Luyện tập tạo các route RESTful API với Express và xử lý dữ liệu cơ bản.

Tóm tắt nội dung

Tạo một API quản lý các câu trích dẫn bằng mảng dữ liệu nội bộ (chưa kết nối DB).

Yêu cầu

- CRUD cơ bản (chỉ READ và CREATE).
- Giao tiếp bằng JSON.

Các chức năng

- GET /quotes – trả tất cả câu trích dẫn.
- GET /quotes/:id – trả một câu cụ thể.
- POST /quotes – thêm câu mới.

Công nghệ sử dụng

- Node.js
- Express.js
- Postman (để test API)

Cấu trúc dự án

```
quotes-api/  
├── package.json  
└── server.js
```

Các bước thực hiện

- Khởi tạo project với `npm init`.
- Cài Express.
- Tạo mảng `quotes` mẫu.
- Viết các route xử lý GET/POST.
- Test API với Postman.

Kết quả cần đạt được

- Viết được các route cơ bản với Express.
- API trả dữ liệu JSON đúng định dạng và endpoint hoạt động đúng.

Test case:

1. Gửi GET request tới /quotes → Nhận mảng JSON chứa các câu trích dẫn mẫu.
2. Gửi POST request tới /quotes với dữ liệu:

json

```
{  
  "author": "Albert Einstein",  
  "quote": "Life is like riding a bicycle. To keep your balance you must keep moving."  
}
```

→ Nhận lại đối tượng JSON chứa câu quote vừa được thêm.

BÀI TẬP 3: Task CRUD API với MongoDB

Mục đích

Áp dụng kỹ năng CRUD với MongoDB thông qua Express và Mongoose.

Tóm tắt nội dung

Tạo RESTful API quản lý task (title, description, status) và lưu vào MongoDB.

Yêu cầu

- Kết nối MongoDB qua Mongoose.
- Cung cấp các route CRUD đầy đủ.

Các chức năng

- GET /tasks
- GET /tasks/:id
- POST /tasks
- PUT /tasks/:id
- DELETE /tasks/:id

Công nghệ sử dụng

- Node.js
- Express.js
- MongoDB
- Mongoose

Cấu trúc dự án

```
task-api/  
├── models/  
│   └── Task.js  
├── routes/  
│   └── tasks.js  
├── server.js  
└── package.json
```

Các bước thực hiện

- Khởi tạo project và cài dependencies: `express`, `mongoose`, `nodemon`.
- Tạo model `Task`.
- Tạo routes và xử lý các method CRUD.
- Kết nối MongoDB với Mongoose.
- Test API qua Postman.

Kết quả cần đạt được

- Viết được API CRUD đầy đủ cho model `Task`.
- Lưu và truy xuất dữ liệu từ MongoDB qua Mongoose.
- Áp dụng mô hình MVC trong tổ chức mã nguồn.

Test case:

1. Gửi POST request tới `/tasks` với dữ liệu:

json

```
{  
  "title": "Learn Express",  
  "description": "Build CRUD app",  
  "status": "pending"  
}
```

→ Trả về task mới với `_id` MongoDB.

2. Gửi GET request tới `/tasks` → Nhận mảng các task, có chứa task vừa tạo.

BÀI TẬP 4: Logger Middleware

Mục đích

Hiểu luồng middleware trong Express bằng cách viết và tích hợp middleware tùy chỉnh.

Tóm tắt nội dung

Tạo middleware ghi log mỗi khi có request gửi tới server.

Yêu cầu

- In ra method và URL của request.

Các chức năng

- Middleware logger cho toàn bộ route.

Công nghệ sử dụng

- Node.js
- Express.js

Cấu trúc dự án

```
middleware-demo/  
├── server.js  
└── middleware/  
    └── logger.js
```

Các bước thực hiện

- Tạo file `logger.js`.
- Tạo middleware function.
- Gắn middleware vào server.
- Tạo vài route để test logger.

Kết quả cần đạt được

- Mỗi lần có request, logger middleware hiển thị thông tin method và URL.
- Middleware hoạt động trên toàn bộ ứng dụng.

Test case:

1. Gửi request tới `/hello` → Terminal log: `GET /hello`.
2. Gửi POST request tới `/test` → Terminal log: `POST /test`.

BÀI TẬP 5: Express Generator Project

Mục đích

Làm quen với cách tạo project chuyên nghiệp bằng Express Generator.

Tóm tắt nội dung

Tạo app Express tự động có cấu trúc thư mục rõ ràng và tích hợp MongoDB.

Yêu cầu

- Sử dụng `express-generator`.
- Cấu trúc MVC rõ ràng.
- Tạo route `/users` với model MongoDB.

Các chức năng

- GET /users, POST /users dùng MongoDB.

Công nghệ sử dụng

- Express Generator
- MongoDB
- Mongoose

Cấu trúc dự án

Tự động tạo như:

cpp

```
myapp/  
├─ app.js  
├─ routes/  
├─ models/  
├─ public/  
└─ views/
```

Các bước thực hiện

- Cài express-generator toàn cục.
- Chạy `express myapp --no-view`.
- Cài mongoose.
- Tạo model User.
- Kết nối MongoDB.
- Viết route CRUD cho /users.

Kết quả cần đạt được

- Khởi tạo được app với cấu trúc chuyên nghiệp.
- Tích hợp MongoDB và tạo route /users hoạt động đúng.
- Hiểu luồng xử lý theo mô hình MVC.

Test case:

1. Gửi POST request tới /users với body:

json

```
{  
  "name": "John Doe",  
  "email": "john@example.com"  
}
```

→ Trả về thông tin user mới và `_id`.

2. Gửi GET request tới /users → Trả về mảng chứa user vừa tạo.

TỪ ĐIỂN THUẬT NGỮ (FOUNDATION GLOSSARY)

KIẾN TRÚC & PATTERNS

Thuật ngữ	Định nghĩa	Ví dụ/Tương tự tiếng Việt
Middleware	Phần mềm trung gian xử lý request/response trong ứng dụng web	"Bộ lọc xử lý" - như lớp kiểm tra an ninh trước khi vào tòa nhà
REST (Representational State Transfer)	Kiến trúc cho hệ thống phân tán, dùng HTTP methods để thao tác tài nguyên	"Chuẩn giao tiếp API" - như menu nhà hàng có sẵn món, khách gọi món
CRUD (Create-Read-Update-Delete)	4 thao tác cơ bản trên dữ liệu	"Thêm-Xem-Sửa-Xóa" - chức năng cơ bản của mọi phần mềm
Routing	Xác định endpoint nào xử lý request nào	"Bảng chỉ đường" - như số phòng trong khách sạn
Endpoint	URL cụ thể nhận request	"Điểm đến" - như địa chỉ email: user@gmail.com
API (Application Programming Interface)	Giao diện lập trình ứng dụng	"Cổng giao tiếp" - như ổ cắm điện: chuẩn chung để kết nối
JSON (JavaScript Object Notation)	Định dạng trao đổi dữ liệu nhẹ	"Văn bản có cấu trúc" - như form đăng ký có các ô thông tin
HTTP Methods	GET, POST, PUT, PATCH, DELETE	"Động từ hành động" - như lệnh: lấy, gửi, thay toàn bộ, sửa một phần, xóa

DATABASE & NOSQL

Thuật ngữ	Định nghĩa	Ví dụ/Tương tự tiếng Việt
NoSQL	Database không dùng SQL, linh hoạt schema	"Kho dữ liệu mềm dẻo" - như tủ đựng đồ linh hoạt
Document Database	Database lưu dữ liệu dạng document (JSON/BSON)	"Hồ sơ lưu trữ" - như hồ sơ bệnh án: mỗi bệnh nhân một tập tin riêng
MongoDB	Document database phổ biến nhất	"Excel cho dữ liệu JSON"
Collection	Nhóm documents trong MongoDB	"Tập hồ sơ" - như folder chứa các file cùng loại
Document	Đơn vị dữ liệu trong MongoDB (JSON-like)	"Hồ sơ cá nhân" - như CV của một người
BSON (Binary JSON)	Định dạng nhị phân của JSON	"JSON nén" - như file ZIP của JSON
Schema	Cấu trúc của dữ liệu	"Khuôn mẫu" - như form mẫu có sẵn các ô cần điền
Embedded Document	Document nằm trong document khác	"Hộp trong hộp" - như địa chỉ nằm trong hồ sơ cá nhân

EXPRESSJS SPECIFICS

Thuật ngữ	Định nghĩa	Ví dụ/Tương tự tiếng Việt
Express Generator	Công cụ tạo khung ứng dụng Express tự động	"Máy in khung nhà" - tạo bộ khung cơ bản để xây nhà
app.use()	Phương thức thêm middleware vào ứng dụng	"Lắp thêm bộ phận" - như lắp camera an ninh vào hệ thống
req (Request)	Đối tượng chứa thông tin đến từ client	"Bưu kiện đến" - có thông tin người gửi, nội dung
res (Response)	Đối tượng để gửi phản hồi về client	"Bưu kiện đi" - cần đóng gói và gửi đi
Router	Đối tượng quản lý nhóm routes	"Bộ định tuyến con" - như tầng trong tòa nhà có nhiều phòng
Template Engine	Công cụ render HTML với dữ liệu động	"Máy in thông minh" - in form có chỗ trống điền số liệu
Static Files	File tĩnh (HTML, CSS, JS, ảnh)	"Đồ đạc cố định" - như bàn ghế, tranh treo tường

Body Parser	Middleware đọc dữ liệu từ request body	"Máy quét nội dung" - như máy scan thư đến
-------------	--	--

RESTFUL ROUTING

Thuật ngữ	Định nghĩa	Ví dụ/Tương tự tiếng Việt
RESTful	Tuân theo nguyên tắc REST	"Chuẩn mực REST" - như lái xe đúng luật giao thông
Resource	Đối tượng/thực thể trong hệ thống	"Tài nguyên" - như sản phẩm, người dùng, đơn hàng
URI (Uniform Resource Identifier)	Định danh duy nhất cho resource	"Mã số định danh" - như số CMND, mã sản phẩm
HTTP Status Codes	Mã trạng thái phản hồi HTTP	"Mã thông báo kết quả" - như 200: OK, 404: Không tìm thấy
Query Parameters	Tham số truy vấn trong URL (?key=value)	"Điều kiện lọc" - như tìm sản phẩm: ?category=điện thoại&price<1000
Route Parameters	Biến động trong URL (/users/:id)	"Chỗ trống trong địa chỉ" - như phòng số :sophong
Request Body	Dữ liệu gửi kèm request (POST/PUT)	"Nội dung gói hàng" - như thông tin đăng ký gửi trong thư
Headers	Thông tin meta của request/response	"Tiêu đề phụ" - như thông tin người gửi trên bì thư

MIDDLEWARE TYPES

Thuật ngữ	Định nghĩa	Ví dụ/Tương tự tiếng Việt
Application-level Middleware	Áp dụng cho toàn bộ app	"Cổng kiểm soát chung" - như cổng an ninh vào tòa nhà
Router-level Middleware	Áp dụng cho nhóm routes cụ thể	"Cổng kiểm soát tầng" - như kiểm soát vào từng tầng riêng
Error-handling Middleware	Xử lý lỗi trong ứng dụng	"Phòng y tế" - xử lý khi có sự cố xảy ra
Built-in Middleware	Middleware có sẵn trong Express	"Trang bị tiêu chuẩn" - như hệ thống PCCC có sẵn
Third-party Middleware	Middleware từ bên thứ ba	"Trang bị mua thêm" - như camera an ninh hãng khác
Custom Middleware	Middleware tự viết	"Trang bị tự chế" - như hệ thống báo động tự lắp

DEPLOYMENT & TOOLS

Thuật ngữ	Định nghĩa	Ví dụ/Tương tự tiếng Việt
npm (Node Package Manager)	Công cụ quản lý thư viện Node.js	"App Store cho code" - như CH Play cho Android
package.json	File cấu hình dự án Node.js	"Hồ sơ dự án" - như hợp đồng có thông tin dự án
Dependencies	Thư viện bên ngoài cần cài đặt	"Vật tư mua ngoài" - như xi măng, gạch mua để xây nhà
DevDependencies	Thư viện chỉ dùng cho phát triển	"Công cụ thi công" - như máy trộn bê tông (chỉ dùng khi xây)
Mongoose	ODM (Object Document Mapper) cho MongoDB	"Phiên dịch viên" - dịch giữa JavaScript và MongoDB
ORM/ODM	Công cụ ánh xạ đối tượng với database	"Cầu nối" - như adapter chuyển đổi cổng
MongoDB Atlas	MongoDB dịch vụ đám mây	"MongoDB thuê bao" - như thuê máy chủ trên mây

MongoDB Compass	GUI để quản lý MongoDB	"Bảng điều khiển" - như remote điều khiển TV
-----------------	------------------------	--

CÁC KHÁI NIỆM QUAN TRỌNG KHÁC

Thuật ngữ	Định nghĩa	Ví dụ/Tương tự tiếng Việt
Callback Function	Hàm được truyền như tham số, gọi sau khi hoàn thành	"Hẹn giờ gọi lại" - như đặt lịch hẹn tái khám
Async/Await	Cú pháp xử lý bất đồng bộ	"Chờ cho xong rồi làm tiếp" - như xếp hàng chờ đến lượt
Promise	Đối tượng đại diện cho kết quả tương lai	"Phiếu hẹn" - như số thứ tự ở bệnh viện
Environment Variables	Biến môi trường cho cấu hình	"Cài đặt theo môi trường" - như điều hòa nhiệt độ khác cho từng phòng
CORS (Cross-Origin Resource Sharing)	Chia sẻ tài nguyên giữa các domain khác nhau	"Giấy thông hành" - cho phép domain khác truy cập tài nguyên
JWT (JSON Web Token)	Token xác thực dạng JSON	"Thẻ ra vào" - như thẻ nhân viên có thông tin mã hóa
Session	Phiên làm việc giữa client và server	"Phiên họp" - như cuộc họp có thời gian bắt đầu/kết thúc
Cookie	Dữ liệu nhỏ lưu ở trình duyệt	"Giấy ghi nhớ" - như tờ ghi chú dán trên bàn

PHÂN LOẠI THEO CẤP ĐỘ HIỂU BIẾT

Level 1: Cơ bản (Must Know)

- Middleware, Routing, CRUD, JSON, API
- GET/POST/PUT/DELETE, Status Codes
- MongoDB, Collection, Document
- npm, package.json, dependencies

Level 2: Trung cấp (Should Know)

- RESTful, Resource, URI
- Mongoose, ODM, Schema
- Environment Variables, CORS
- Template Engine, Static Files

Level 3: Nâng cao (Good to Know)

- Promise, Async/Await, Callback
- JWT, Session, Authentication
- Error-handling Middleware
- Query Parameters vs Route Parameters

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

Category 1: Middleware - Phân tích sâu về kiến trúc

1. Phân tích chiến lược Middleware Composition

"Hãy phân tích và so sánh 3 pattern sắp xếp middleware trong Express: linear chain (app.use()), modular routers, và middleware factories. Với mỗi pattern, xây dựng một case study cụ thể trong production (ví dụ: e-commerce, social media, banking) để minh họa ưu/nhược điểm. Đánh giá performance impact của mỗi approach khi số lượng middleware tăng lên 50+."

2. Đánh giá lựa chọn Error Handling Strategy

"Xây dựng một ma trận đánh giá (evaluation matrix) để lựa chọn error handling strategy cho Express application. Phân tích 4 approaches: centralized error middleware, domain-based errors, async error wrappers, và error subclassing. Mỗi strategy cần được đánh giá trên 5 tiêu chí: maintainability, performance, debugging ease, testability, và security implications. Cung cấp benchmark cụ thể cho mỗi approach."

3. Phân tích Security Middleware Implementation

"Phân tích sâu implementation của 5 security middleware phổ biến: helmet, csrf, rate-limiter, xss-clean, và cors. Với mỗi middleware: (a) Deconstruct cơ chế hoạt động, (b) Phân tích potential bypass techniques, (c) Đề xuất hardening strategies, (d) Benchmark performance overhead. Tạo một threat model cho ứng dụng Express và map các middleware vào từng threat."

Category 2: RESTful API Design - Đánh giá kiến trúc

4. Phân tích REST Maturity Model (RMM) áp dụng

"Áp dụng Richardson Maturity Model để đánh giá 3 real-world public APIs (GitHub API, Stripe API, Twitter API). Phân tích từng level (0-3) và đánh giá business/technical trade-offs của việc đạt đến level 3 (HATEOAS). Xây dựng một cost-benefit analysis cho việc implement HATEOAS trong một startup vs enterprise application."

5. Đánh giá GraphQL vs REST cho microservices

"Xây dựng một comparative analysis framework để đánh giá GraphQL vs REST trong microservices architecture. Phân tích 6 dimensions: performance (N+1 problem), caching strategy, versioning approach, developer experience, security model, và operational complexity. Cung cấp decision tree để help choose between technologies based on specific use cases."

6. Phân tích API Versioning Strategies trong enterprise

"Phân tích sâu 4 API versioning strategies: URI versioning (/v1/resource), header versioning (Accept-version), parameter versioning (?version=1), và content negotiation. Mỗi strategy cần được đánh giá trên: client adoption cost, backward compatibility, deprecation management, và tooling support. Tạo một migration plan từ v1 sang v2 cho API có 10 triệu active users."

Category 3: Express Generator & Scaffolding - Phân tích production patterns

7. Đánh giá Enterprise Project Structure Patterns

"Phân tích và so sánh 3 enterprise Express project structures: layered architecture (controller-service-repository), clean architecture (use cases), và vertical slice architecture. Với mỗi pattern: (a) Diagram flow của một request, (b) Tính toán complexity metrics (cyclic dependencies, coupling), (c) Đo lường impact on testing, (d) Đánh giá team scalability. Cung cấp migration guide giữa các patterns."

8. Phân tích Build Optimization cho Express applications

"Xây dựng một comprehensive build optimization strategy cho Express application từ development đến production. Phân tích: bundle optimization (webpack vs esbuild), tree shaking strategies, environment-specific builds, Docker multi-stage builds, và cold start optimization. Tạo một performance budget và monitoring dashboard cho build process."

9. Đánh giá Monorepo vs Polyrepo cho large codebases

"Phân tích architectural trade-offs của monorepo và polyrepo cho Express applications với 50+ microservices. Đánh giá trên: dependency management, CI/CD pipeline complexity, team autonomy, code sharing efficiency, và deployment strategies. Cung cấp case study của 2 công ty đã chọn mỗi approach và phân tích kết quả."

Category 4: NoSQL & MongoDB - Phân tích database design

10. Phân tích Data Modeling Patterns cho MongoDB

"Phân tích 4 data modeling patterns trong MongoDB: embedded documents, references, hybrid approach, và bucket pattern. Với mỗi pattern: (a) Thiết kế schema cho e-commerce application, (b) Phân tích read/write performance với 1M documents, (c) Đánh giá query complexity, (d) Phân tích migration strategies khi pattern thay đổi. Cung cấp decision framework để chọn pattern."

11. Đánh giá Transaction Management trong distributed MongoDB

"Phân tích sâu distributed transactions trong MongoDB (multi-document transactions). Đánh giá: performance overhead vs consistency guarantees, retry strategies, deadlock detection, và monitoring patterns. Thiết kế một Saga pattern implementation cho e-commerce order processing và so sánh với native MongoDB transactions."

12. Phân tích Index Optimization Strategy

"Phân tích và đánh giá comprehensive index optimization strategy. Bao gồm: (a) Compound index design patterns, (b) Partial vs sparse indexes, (c) Text search index optimization, (d) Geospatial index patterns, (e) TTL indexes for time-series data. Cung cấp một index analysis toolkit và query performance prediction model."

Category 5: Advanced Patterns & Performance

13. Phân tích Caching Strategy cho high-traffic APIs

"Phân tích multi-layer caching strategy cho REST API với 10K requests/second. Đánh giá: in-memory caching (Redis), database query caching, CDN caching, và browser caching. Phân tích cache invalidation strategies, cache stampede prevention, và consistency models. Thiết kế một cache coherence protocol cho distributed Express applications."

14. Đánh giá Real-time Communication Patterns

"Phân tích và so sánh 4 real-time communication patterns trong Express: WebSockets, Server-Sent Events (SSE), HTTP/2 Server Push, và long polling. Mỗi pattern cần được

đánh giá trên: scalability, browser compatibility, message reliability, và development complexity. Thiết kế một hybrid solution cho collaborative editing application."

15. Phân tích Observability trong Microservices

"Xây dựng một comprehensive observability framework cho Express microservices. Phân tích 3 pillars: logging (structured vs unstructured), metrics (business vs technical), và tracing (distributed tracing implementation). Đánh giá các tools: OpenTelemetry, Prometheus, Grafana, và ELK stack. Thiết kế một correlation ID strategy cho end-to-end request tracking."

10 PROMPT TỰ NGHIÊN CỨU THỰC TIỄN CHUYÊN NGHIỆP

🔗 VẤN ĐỀ THỰC TẾ TRONG PRODUCTION

Category 1: Production Middleware - Bài học từ incident thực tế

1. Middleware Chain Debugging trong Microservices Distributed System

"Bạn đang maintain hệ thống 15 microservices Express với 200+ middleware. Một incident xảy ra: request timeout sau 30s nhưng không có error logs. Thiết kế một distributed middleware tracing system với: (a) Correlation ID propagation qua HTTP headers, message queues, database calls (b) Middleware execution timing per service (c) Visual dependency graph generation. Implement solution sử dụng OpenTelemetry và phân tích chi phí performance overhead."

2. Graceful Shutdown Pattern cho High-Traffic APIs

"API của bạn nhận 10K RPM và cần deploy nhiều lần/ngày. Mỗi lần deploy, 5% requests bị fail với 502 errors. Thiết kế graceful shutdown strategy bao gồm: health check endpoints, connection draining, in-flight request completion, và database connection pooling cleanup. Benchmark thời gian shutdown với/không có strategy và tính toán business impact (lost revenue)."

3. Security Middleware Configuration Management

"Team bạn có 5 environments (dev, staging, uat, prod, dr) với security middleware configuration khác nhau. Một developer vô tình deploy staging config lên production làm lộ CORS settings. Thiết kế một security configuration management system với: environment-specific validation, pre-deployment security scanning, audit trail, và automatic rollback triggers."

Category 2: REST API ở Scale - Scaling challenges

4. Rate Limiting Strategies cho Public APIs với 1M+ Users

"Public API của công ty bị DDoS attack làm sập service. Phân tích và implement multi-layer rate limiting: (a) Global IP-based limiting (NGINX) (b) User-based token bucket (Redis) (c) Business logic limiting (per feature). Đánh giá trade-offs: false positives với shared IPs, impact đến legitimate high-volume customers, và cost của Redis cluster."

5. API Versioning và Deprecation trong Enterprise

"Công ty có 50+ internal teams sử dụng API của bạn. Cần deprecate version 1.0 sau 2 năm với 1000+ active clients. Thiết kế deprecation strategy: automated client detection, usage analytics dashboard, migration assistant tools, và communication plan. Tính toán effort estimation và risk assessment cho từng phase."

6. Bulk Operations và Async Processing Pattern

"API POST /users nhận batch 10K users nhưng timeout sau 60s. Thiết kế async bulk processing pattern: request validation → job queue (RabbitMQ) → worker processing → webhook/callback notification. Phân tích trade-offs: data consistency, partial failures handling, retry logic, và monitoring dashboard."

Category 3: Express trong Microservices Architecture

7. Shared Middleware Library Management

"10 teams trong công ty đều tự implement authentication middleware với bugs và security holes khác nhau. Thiết kế một internal npm package với: (a) Standardized auth middleware (JWT, OAuth2) (b) Configuration management (c) Version compatibility matrix (d) Automated security updates. Đề xuất governance model và rollout strategy."

8. Cross-Cutting Concerns: Logging, Metrics, Tracing

"Mỗi service logs theo format khác nhau, không thể correlate requests across services. Implement unified observability stack: structured JSON logging với correlation IDs, Prometheus metrics collection, Jaeger distributed tracing. Tính toán storage requirements cho 1TB logs/day và cost optimization strategies."

9. Database Connection Pooling trong Serverless/Lambda

"Express apps chạy trên AWS Lambda bị database connection exhaustion do cold starts. Thiết kế connection management strategy: connection pool per container, proxy (PgBouncer/RDS Proxy), và connection reuse patterns. Benchmark cold start impact với connection pooling và không có pooling."

Category 4: MongoDB Production Challenges

10. Schema Migration Strategy với Zero Downtime

"Cần thay đổi schema cho collection với 100M documents đang được query real-time. Thiết kế zero-downtime migration: dual-write pattern, backward-compatible schema design, data validation scripts, và rollback plan. Tính toán migration duration, performance impact, và risk assessment."

11. Sharding Strategy cho Time-Series Data

"IoT application ghi 10K documents/second. Thiết kế sharding strategy: time-based shard key, hot shard problem mitigation, archival policy, và query routing optimization. Phân tích trade-offs giữa range-based vs hashed sharding."

12. MongoDB Transaction Performance Optimization

"E-commerce checkout process sử dụng multi-document transactions nhưng performance giảm 40%. Phân tích và optimize: transaction timeout settings, retry logic với exponential backoff, read/write concern levels, và alternative patterns (compensating transactions)."

Category 5: DevOps và SRE Practices

13. Canary Deployment và Traffic Shifting

"Mỗi lần deploy API mới, 0.1% users gặp bugs nhưng bị report muộn. Thiết kế canary deployment pipeline: traffic splitting (5%, 25%, 50%, 100%), automated rollback based on metrics (error rate, latency), và user segmentation (internal users first). Implement với service mesh (Istio) hoặc API gateway."

14. Configuration Management cho Multi-Region Deployment

"API deploy ở 3 regions (US, EU, Asia) với configuration khác nhau: database endpoints, third-party API keys, feature flags. Thiết kế configuration management: environment variables, config files, secret management (Hashicorp Vault/AWS Secrets Manager), và configuration validation. Đảm bảo consistency và security."

15. Disaster Recovery và Chaos Engineering

"Primary region bị outage, cần failover trong 5 phút. Thiết kế DR strategy: multi-region MongoDB replica sets, global load balancer health checks, DNS failover, và data consistency verification. Thiết lập chaos experiments để test recovery procedures."

CASE STUDIES THỰC TẾ TỪ TECH COMPANIES

Case 1: Uber's Express Middleware Architecture

Challenge: 4000+ microservices, global scale

Solution:

javascript

// Middleware standardization

```
class UberMiddleware {  
  static auth() { /* JWT + OAuth2 + rate limiting */ }  
  static tracing() { /* OpenTracing + correlation IDs */ }  
  static validation() { /* Schema validation + sanitization */ }  
}
```

// Deployment pattern: canary + automated rollback

Research Prompt: "Phân tích Uber's middleware architecture: làm thế nào họ maintain consistency across 4000+ services? Implement similar framework với 10 services và đo lường maintenance overhead."

Case 2: Netflix's API Gateway Evolution

Challenge: 800+ microservices, diverse clients

Solution: Zuul API gateway với dynamic routing, filtering, monitoring

Research Prompt: "Build simplified API gateway với Express: request routing, response aggregation, circuit breaking, và caching. So sánh performance với Netflix's solution."

Case 3: Airbnb's MongoDB Sharding Strategy

Challenge: Listings data với global queries

Solution: Geo-based sharding + composite indexes

Research Prompt: "Thiết kế sharding strategy cho e-commerce với users từ multiple countries. Benchmark query performance với different shard key strategies."